

TRÍ TUỆ NHÂN TẠO • HỌC KỲ II - 2026

# BÀI TOÁN NGƯỜI GIAO HÀNG

SINH VIÊN THỰC HIỆN

MÃ SỐ SINH VIÊN

Lê Văn Đức

23021248



PHẦN I

# TỔNG QUAN & THÁCH THỨC TỔ HỢP



# Bài toán TSP là gì?

## Định nghĩa cơ bản

TSP (Traveling Salesman Problem) yêu cầu tìm một lộ trình ngắn nhất đi qua một danh sách các thành phố, mỗi nơi đúng một lần và quay về điểm xuất phát.

Trong lý thuyết đồ thị, đây là bài toán tìm \*\*Chu trình Hamiltonian\*\* có trọng số nhỏ nhất trên một đồ thị đầy đủ.

*"Bài toán tưởng chừng đơn giản nhưng lại là nền tảng cho tối ưu hóa logistics, lập trình sản xuất và thiết kế vi mạch."*

 TSP Path Map



# Sự bùng nổ tổ hợp

$$(n-1)! / 2$$

SỐ LỘ TRÌNH KHẢ THI

 Factorial Complexity Graph

## Thách thức NP-Hard

Với  $n$  thành phố, số lượng hoán vị tăng theo hàm giai thừa. Đây là lý do tại sao phương pháp "vét cạn" (Brute Force) trở nên bất khả thi ngay cả với siêu máy tính.

- ▶ 10 thành phố: 181,440 lộ trình.
- ▶ 20 thành phố:  $1.2 \times 10^{18}$  lộ trình.



# Các hướng tiếp cận trong AI



## Exact Search

Brute Force, Quy hoạch động.  
Đảm bảo tối ưu nhưng chậm ( $O(n!)$ ).



## Informed Search

Sử dụng hàm Heuristic ( $A^*$ , Branch & Bound) để thu hẹp không gian tìm kiếm.



## Metaheuristics

Giải thuật di truyền, kiến, luyện kim.  
Tìm lời giải "đủ tốt" cho bài toán lớn.



PHẦN II

# CHIẾN LƯỢC TÌM KIẾM A\*



# Hàm Heuristic trong TSP

## Hàm Đánh giá $f(n)$

$$f(n) = g(n) + h(n)$$

- $g(n)$ : Chi phí thực tế đã đi từ điểm xuất phát.
- $h(n)$ : Ước lượng chi phí tối thiểu để đi qua các thành phố còn lại.

## Tính Admissibility

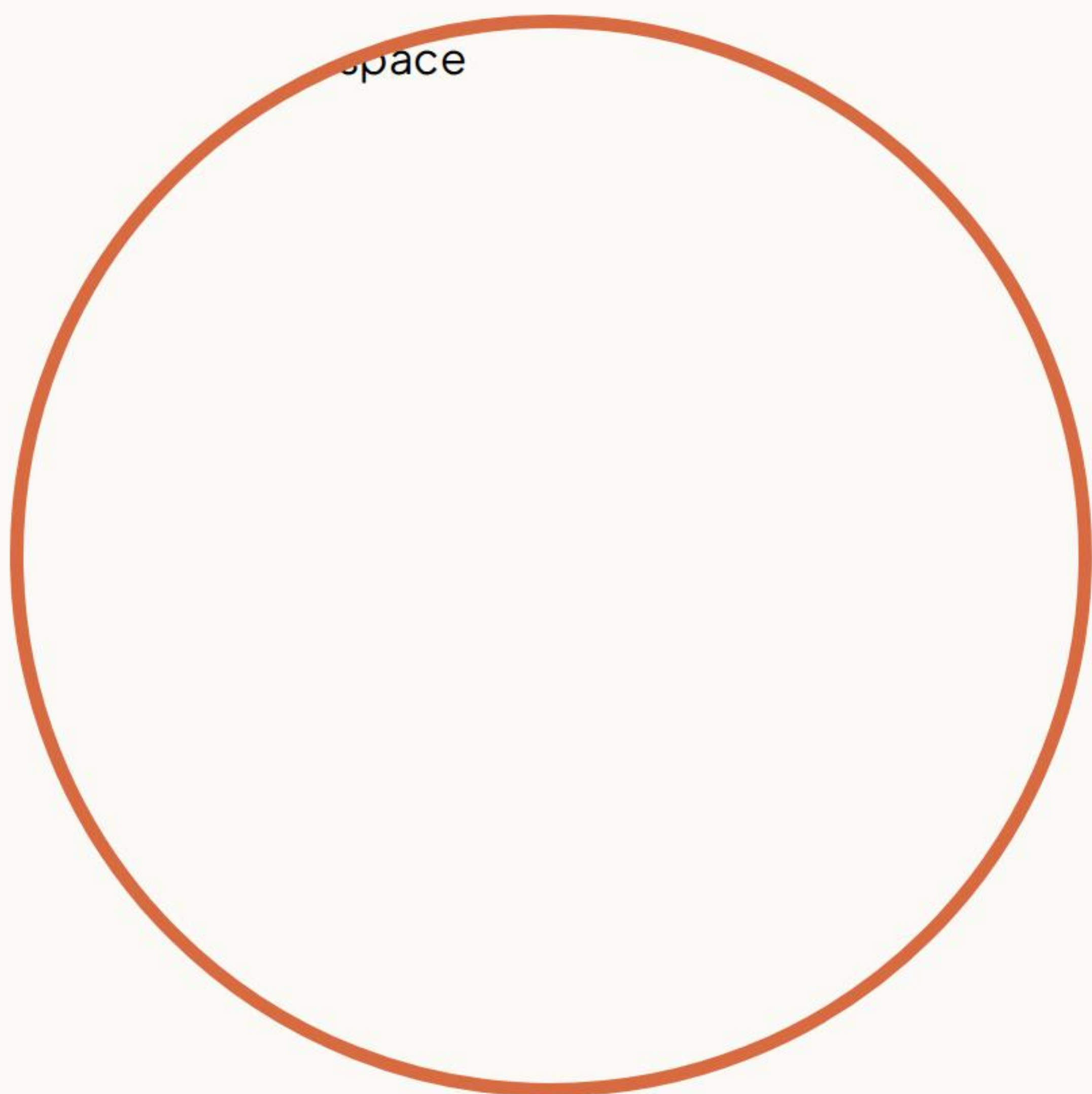
Để A\* tìm được lời giải tối ưu,  $h(n)$  phải không bao giờ lớn hơn chi phí thực tế.

Các kỹ thuật ước lượng: Cây khung tối thiểu (MST), 1-Tree, hoặc tổng các cạnh ngắn nhất.

 MST Visualization



# Sử dụng A\* để giải bài toán TSP



## Triển khai thực tế

Mình đã hiện thực hóa thuật toán A\* bằng ngôn ngữ Python, sử dụng cấu trúc dữ liệu Priority Queue để tối ưu việc chọn nút tiếp theo.

```
# Trạng thái trong Priority Queue heapq.heappush(pq, (g + h, g, path, unvisited))
```

\*Đây là một ví dụ minh họa cách lý thuyết A\* được áp dụng vào mã lệnh thực tế.



# Phân tích Logic mã nguồn

-  **Hàm Heuristic  $h/2$ :** Tính tổng các cạnh ngắn nhất của các thành phố chưa đi qua và chia 2 để đảm bảo tính Admissible (do mỗi cạnh được tính 2 lần).
-  **Cấu trúc Heap:** Sử dụng `heapq` giúp việc lấy trạng thái có  $f(n)$  nhỏ nhất đạt độ phức tạp  $O(\log N)$ .
-  **Mở rộng trạng thái:** Duyệt qua tập `unvisited` để tạo ra các lộ trình mới, liên tục cập nhật khoảng cách thực tế  $g$ .
-  **Điều kiện dừng:** Khi tập `unvisited` trống, cộng thêm quãng đường quay về `start` để hoàn thành chu trình Hamilton.



# Kết quả chạy thử nghiệm

| Thông số            | Giá trị đầu vào     | Kết quả tối ưu        |
|---------------------|---------------------|-----------------------|
| Ma trận khoảng cách | 4×4 (Thành phố 0–3) | Đã xác định           |
| Lộ trình tốt nhất   | –                   | [ 0 , 1 , 3 , 2 , 0 ] |
| Tổng chi phí (Cost) | –                   | 80                    |
| Thời gian thực thi  | –                   | < 0.01 giây           |

"Thuật toán của mình đã tìm thấy lộ trình tối ưu 80 đơn vị khoảng cách một cách chính xác."



# Hiệu quả của Heuristic Search

Vét cạn (Brute Force)

Tất cả các nút

A\*

Chỉ xét các nút tiềm năng

Việc sử dụng A\* giúp giảm số lượng nút phải mở rộng đáng kể so với việc duyệt toàn bộ cây trạng thái.

 A\* Progress





Cảm ơn thầy và các bạn đã lắng nghe!

---

**Lê Văn Đức ▪ 23021248 ▪ AI Search Strategy**